

TD01 - Linux and Shell Exercises

Exercises from CM-all-2025

Louis Ledoux

2026-07-05

Table of contents

1 Linux and Shell Exercises	1
1.1 Main Path	1
1.2 Worked Example	2
1.3 Anecdote Or Motivation	2
1.4 Check Question	3
1.5 Backup Index	3
1.6 Backup: TD1: Linux and Shell Exercises	6
1.7 Backup: Ex1: Wildcard characters	6
1.8 Backup: Ex2: Redirections	7
1.9 Backup: Ex3: Redirections	8
1.10 Backup: Ex3: Redirections	9
1.11 Backup: Ex4: Script arguments	10
1.12 Backup: Ex5: Search	11
1.13 Backup: Ex6: Find	12

1 Linux and Shell Exercises

Live pacing: about 8 min

Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

1.1 Main Path

- Start from the question: What does the operating system do between the program and the machine?
- Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- Keep only this slide on the horizontal path during the live lecture.

1.2 Worked Example

- Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- Representative source slide: 101
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Ex6: Find

Describe the effect of the following instructions:
`find ./ -name "tp2" -exec {} < f_in > {} out \;`
`find /var/log -exec grep "www.athabasca.com" {} \;`
`find . -name "rc.conf" -exec chmod o+r {} \;`
`find /usr/local -type f -name "*.html".`

1.3 Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Ex6: Find

```
Describe the effect of the following instructions:
find ./ -name 'tp2' -exec {} < f_in > {} out \;
find /var/log -exec grep 'www.athabasca.com' {} \;
find . -name 'rc.conf' -exec chmod o+r {} \;
find /usr/local -type f -name '*.html'.
```

#101

1.5 Backup Index

Original slides 94-101

- TD1: Linux and Shell Exercises

#94

slide 94

Ex1: Wildcard characters

- List the entries inside the folder /usr/bin whose name starts with the letter m.
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has exactly 3 characters
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has at least 3 characters
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has an extension (postfix after .)

#95

slide 95

Ex2: Redirections

Given the following file hierarchy
user2 is connected at his home folder
Use the command cat:



#96

slide 96

Ex3: Redirections

- Create script with the name execution called with 3 arguments n1 n2 n3 (which are the filenames) performing the following manipulations
- P1 and P2 are programs that read from the stdin and print their result to stdout

- n1 - P1 - n1.r1

- n2 - P1 - n2.r1 - n2.r1 concatenated with n1.r1 - P2 - n3

#97

slide 97

Ex3: Redirections

What happens when the following instruction is executed in the home directory of user1?
execution file1 ../user2/file2 fileN

- usr

- user1 - user2

- file1 - file2 - p1 - p2 - file2

- Write an interactive version of the script execution that asks from the user to provide n1, n2 and n3, instead of passing them as arguments

#98

slide 98

Ex4: Scriptarguments

- Create a script with the name argument using the command expr that prints one by one the arguments passed during the script call
- For instance, the call:
-%argument one two three
- Should print in the stdout
- 3 arguments:
- argument 1 - one
- argument 2 - two
- argument 3 - three

#99

slide 99

anchors: TD1: Linux and Shell Exercises; Ex1: Wildcard characters; Ex2: Redirections; Ex3: Redirections; Ex4: Script arguments; Ex5: Search

1.6 Backup: TD1: Linux and Shell Exercises

Original slide 94

- TD1: Linux and Shell Exercises

#94

1.7 Backup: Ex1: Wildcard characters

Original slide 95

- List the entries inside the folder /usr/bin whose name starts with the letter m.
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has exactly 3 characters

- List the entries inside the folder /usr/bin whose name starts with the letter m and it has at least 3 characters
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has an extension (postfix after .)

Ex1: Wildcard characters

- List the entries inside the folder /usr/bin whose name starts with the letter m.
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has exactly 3 characters
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has at least 3 characters
- List the entries inside the folder /usr/bin whose name starts with the letter m and it has an extension (postfix after .)

#95

1.8 Backup: Ex2: Redirections

Original slide 96

Given the following file hierarchy
 user2 is connected at his home folder
 Use the command cat :
 Create a file f5 with the following content
 5 2 3 -1
 Create a file f6 by concatenating f5 after f4
 Add the file f1 of user1 at the file f6
 Copy the file f2 of user1 as a new file f7 of user2

- usr
- user1
- user2
- f4
- f1
- f2
- f3

Ex2: Redirections

Given the following file hierarchy
user2 is connected at his home folder
Use the command cat:



#96

1.9 Backup: Ex3: Redirections

Original slide 97

- Create script with the name execution called with 3 arguments n1 n2 n3 (which are the filenames) performing the following manipulations
- P1 and P2 are programs that read from the stdin and print their result to stdout
- n1.r1
- P1
- n1
- n3
- P2
- n2.r1 concatenated with n1.r1
- n2.r1
- P1
- n2

Ex3: Redirections

- Create script with the name execution called with 3 arguments n1 n2 n3 (which are the filenames) performing the following manipulations
- P1 and P2 are programs that read from the stdin and print their result to stdout

- n1	- P1	- n1.r1							
								- P2	- n3
- n2	- P1	- n2.r1	- n2.r1 concatenated with n1.r1						

#97

1.10 Backup: Ex3: Redirections

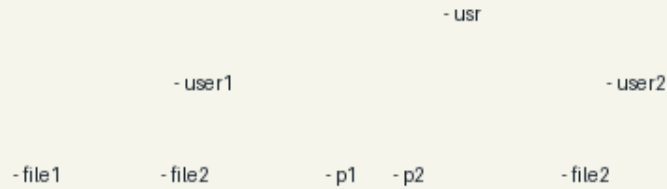
Original slide 98

What happens when the following instruction is executed in the home directory of user1?
execution file1 ../user2/file2 fileN

- usr
- user1
- user2
- file2
- file1
- p2
- file2
- p1
- Write an interactive version of the script execution that asks from the user to provide n1, n2 and n3, instead of passing them as arguments

Ex3: Redirections

What happens when the following instruction is executed in the home directory of user1?
execution file1 ../user2/file2 fileN



- Write an interactive version of the script execution that asks from the user to provide n1, n2 and n3, instead of passing them as arguments

#98

1.11 Backup: Ex4: Script arguments

Original slide 99

- Create a script with the name argument using the command `expr` that prints one by one the arguments passed during the script call.
- For instance, the call:
 - % argument one two three
- Should print in the stdout:
 - 3 arguments :
 - argument 1 - one
 - argument 2 - two
 - argument 3 - three

Ex4: Scriptarguments

- Create a script with the name argument using the command expr that prints one by one the arguments passed during the script call
- For instance, the call:
-%argument one two three
- Should print in the stdout
- 3 arguments:
- argument 1 - one
- argument 2 - two
- argument 3 - three

#99

1.12 Backup: Ex5: Search

Original slide 100

- Create a script with the name mysearch passing two arguments during the call, i.e. motif and filename. The program prints either "filename has motif" or "filename does not have motif"
- Create a script mysearch2 filename that ask the user the motif to be searched inside the filename until the motif end is entered by the user
- Create a script mysearch3 f1 .. fn that performs the same thing as mysearch2, but now it searches inside the files given as arguments.
- /dev/null = trash

Ex5: Search

- Create a script with the name mysearch passing two arguments during the call, i.e. motif and filename. The program prints either "filename has motif" or "filename does not have motif"
- Create a script mysearch2 filename that ask the user the motif to be searched inside the filename until the motif end is entered by the user
- Create a script mysearch3 f1 .. fn that performs the same thing as mysearch2, but now it searches inside the files given as arguments.
- /dev/null = trash

#100

1.13 Backup: Ex6: Find

Original slide 101

Describe the effect of the following instructions:

```
find ./ -name "tp2" -exec {} < f_in > {}.out \;  
find /var/log -exec grep "www.athabasca.com" {} \;  
find . -name "rc.conf" -exec chmod o+r {} \;  
find /usr/local -type f -name "*.html".
```

Ex6: Find

Describe the effect of the following instructions:
find ./ -name "tp2" -exec {} < f_in > {}.out \;
find /var/log -exec grep "www.athabasca.com" {} \;
find . -name "rc.conf" -exec chmod o+r {} \;
find /usr/local -type f -name "*.html".

#101